



VISION-ASSISTED CARTESIAN ROBOTIC SYSTEM FOR 3D BLOCK CONSTRUCTION

Ahmed Gharib
Mohie Halawa
Khalil Qanabita



TABLE OF CONTENTS

**Motivation &
Objectives**

System Design

Bill of Materials

**Vision & Control
System**

**Testing &
Results**

**Conclusion &
Future Work**

BACKGROUND & MOTIVATION

Why This Project?

- Industrial pick-and-place systems must do more than move parts.
- Real assembly cells must verify feeding, gripping, placement, and failures.
- Timing-based systems assume success instead of confirming it.
- The goal is a machine that verifies every stage rather than relying on assumptions.



THE PROBLEM

WHAT AN OPEN-LOOP MACHINE CANNOT SEE

Stacking is sequential: block 7 has to be right or block 18 falls off it. A timing-based machine cannot tell success from failure, so it never stops — it turns one silent failure into a pile.

1

Empty or jammed hopper

The gate opens, no block comes out, the belt runs anyway. The claw closes on nothing.

2

Two blocks instead of one

A gate open far enough for one block is open far enough for two. The second arrives out of sequence.

3

A block that is late or crooked

A belt is not a positioning system. The block is picked by its corner, or missed entirely.

4

Lost steps

Open-loop steppers lose steps against a stall, and nothing in the motor reports it.

5

A hand in the workspace, or a controller reset

A person moves a block, or a USB open reboots the board un-homed. The machine's model is silently wrong.

OBJECTIVES

1

Develop a complete self-feeding robotic cell capable of autonomous 3D block construction.

2

Place blocks at any addressable grid cell and stack level within the machine workspace.

3

Support both vertical and horizontal block orientations through a rotating end-effector and dual-grid system.

4

Detect, localize, and verify blocks using an overhead vision system with calibrated grid mapping.

5

Provide a browser-based platform for structure design, execution, and safe operation of the robotic system.



SCOPE AND BOUNDARIES

WHAT “CLOSED-LOOP” MEANS IN THIS PROJECT

Sensor- and vision-confirmed sequencing — not servo position control.

The machine never verifies a motor shaft. It verifies that a block left the hopper, that a block reached the pickup point, that a build phase began and ended, and that a block is visible where one was commanded. Each of those can refuse permission for the step after it.

IN SCOPE

- CoreXY gantry, Z axis, servo claw, 90° rotation
- Self-feeding module with two ultrasonic confirmations
- Overhead fisheye camera, detection and grid calibration
- Browser console, 3D Build Studio, digital twin
- Two serial protocols and the Pi orchestration

OUT OF SCOPE

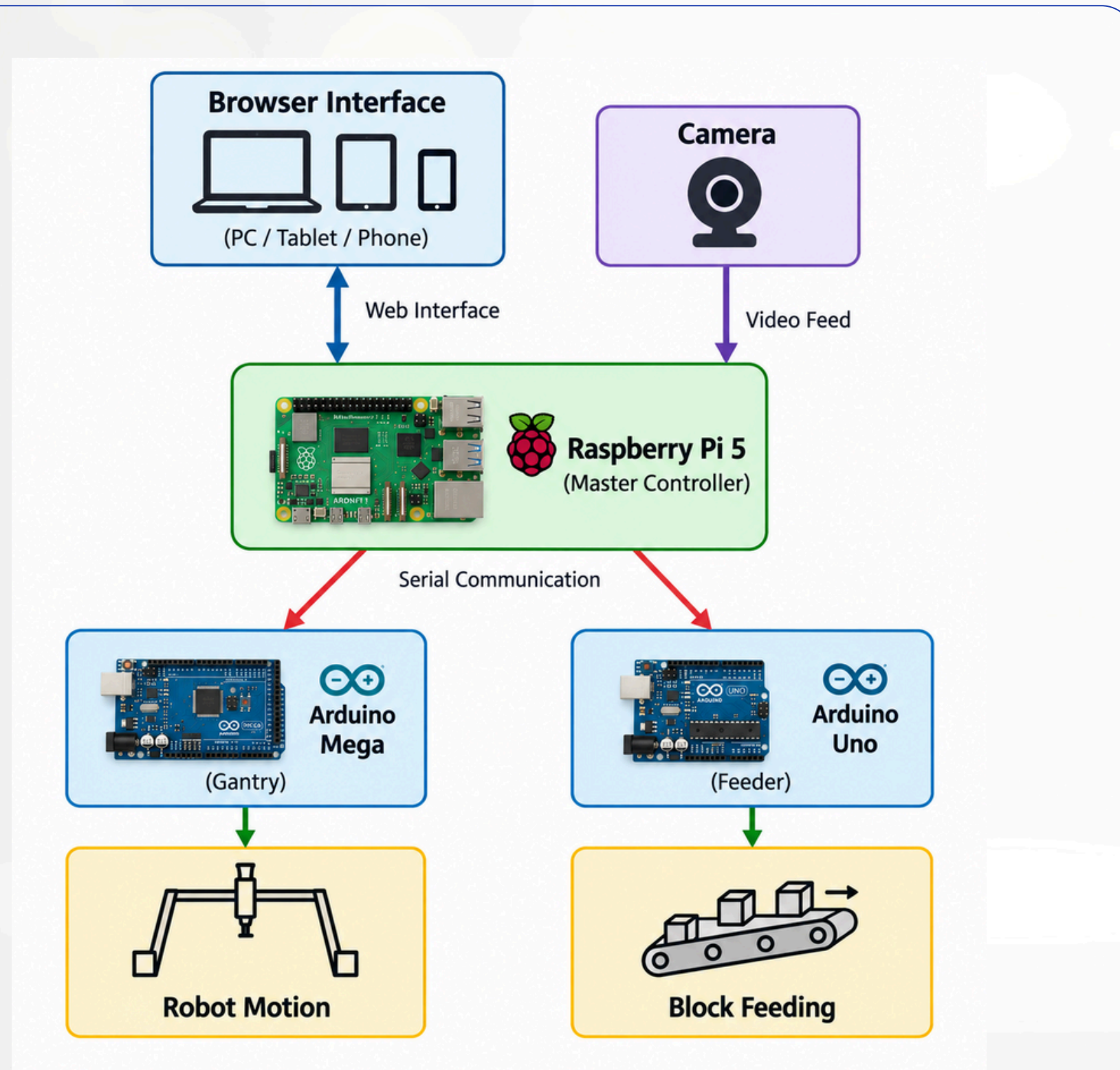
- Autonomous planning — the human designs the structure
- Interrupting a motion already in progress
- Metrological (checkerboard) camera calibration
- Closed-loop motor control — there are no encoders
- Multi-operator accounts and WebRTC video

01 SYSTEM DESIGN

The three-controller architecture, the mechanics, and the components the machine is actually made of.



OVERALL SYSTEM ARCHITECTURE



Computer vision detects blocks and guides the robot to build predefined structures.



THREE CONTROLLERS, ONE MASTER

ONE MASTER, TWO ISOLATED SLAVES

Raspberry Pi 5

8 GB · THE MASTER

- Camera capture and the whole vision pipeline
- FastAPI web service, WebSocket event stream
- The Studio and the browser application
- Two-board orchestration
- Every safety gate and the session lock

Arduino MEGA 2560

THE GANTRY · 9600 8N1

- X, Y and Z step generation and direction
- Homing, hardware and software limits
- Grid-to-steps arithmetic, both grids
- Gripper servo and rotation stepper
- The fourteen-phase build cycle

Arduino Uno

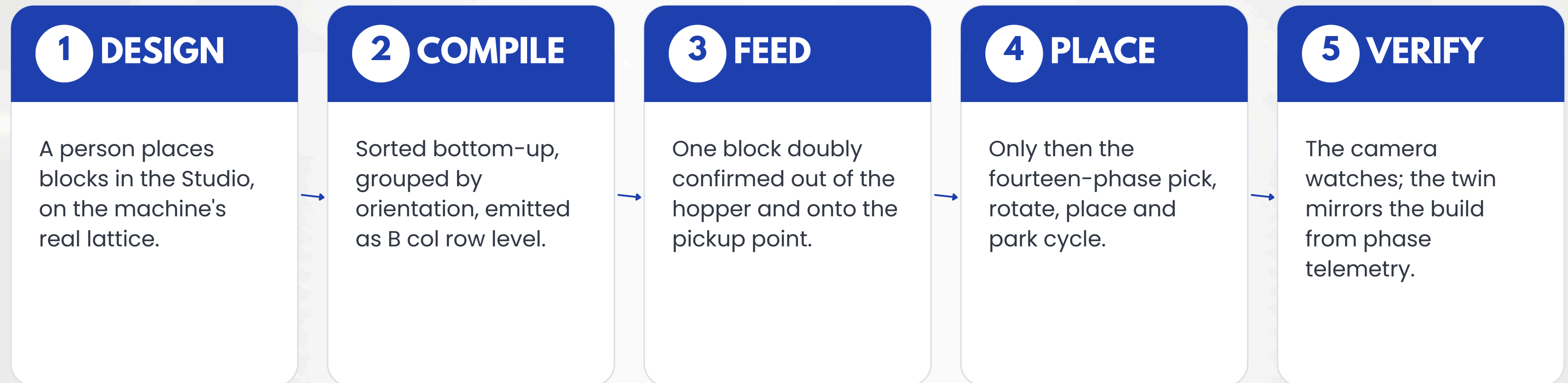
THE FEEDER · 9600 8N1

- Two-stage hopper gate servo
- Conveyor belt stepper (A4988)
- Alignment servo
- Both HC-SR04 ultrasonic sensors
- The complete feed state machine

The two Arduinos have no wire between them and neither can command the other. That isolation is a safety property: permission to pick a block up can only be granted where the master can see it.

HOW ONE BLOCK GETS BUILT

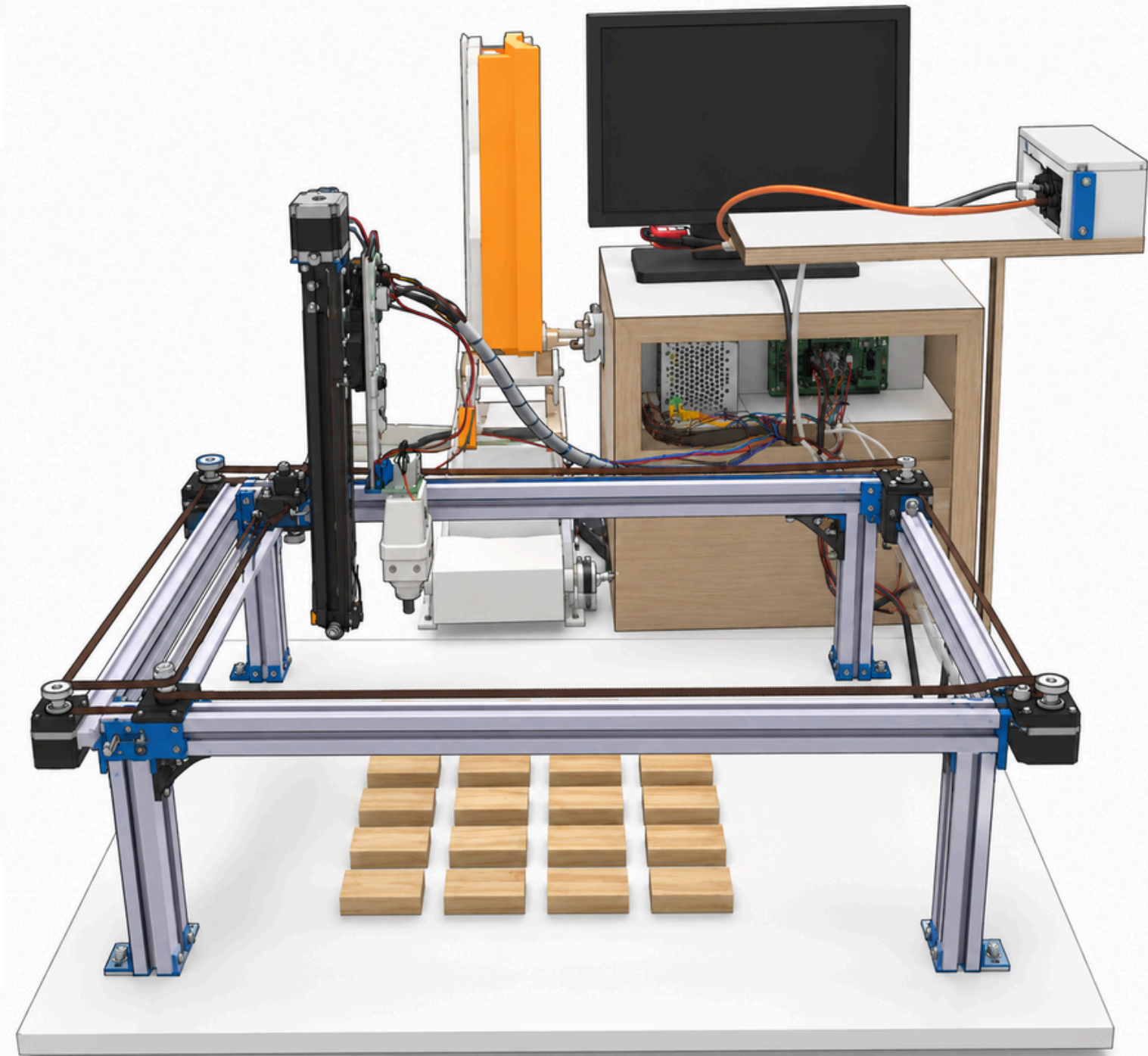
DESIGN › COMPILE › FEED › PLACE › VERIFY



Nothing in this chain is advanced by a timer. Each stage waits for a durable terminal result from the stage before it, and a stage that cannot prove what happened stops the build instead of continuing.

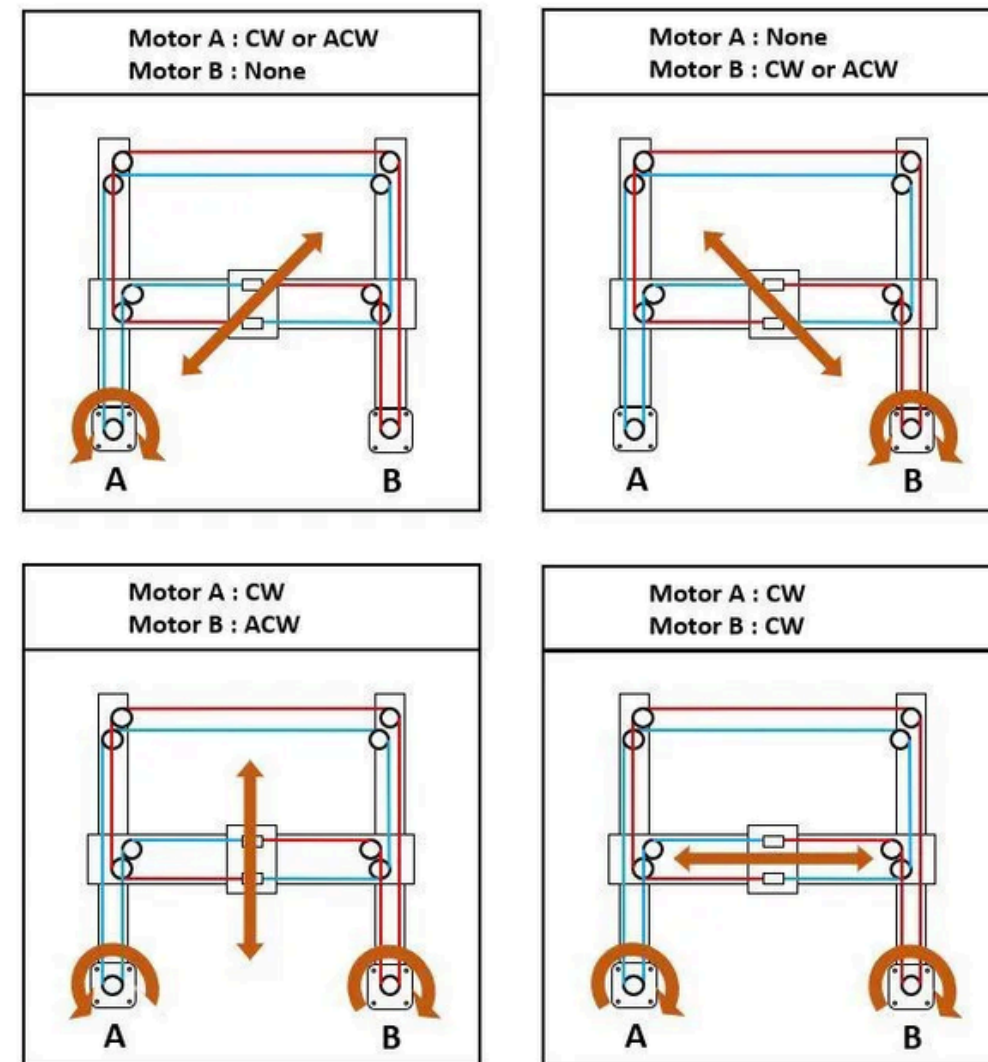
MECHANICAL DESIGN

- X-axis motion system
- Y-axis gantry
- Z-axis actuator
- End-effector (gripper)



THE X/Y STAGE: COREXY

BOTH MOTORS OFF THE MOVING BEAM



Verified on the machine by watching the shafts

CoreXY skeleton and limit-switch homing after Greg06, "Automated Chessboard", Instructables 2022. Z, the claw, the feeder, the camera and all three controllers were built on top of it.

- **Same sense** › the carriage walks along X. Opposed senses › it walks along Y.
- **Two NEMA17 motors** are fixed to the frame and drive one continuous GT2 belt path, so the beam stays light and the envelope stays large.
- **Guidance** is eight castor wheels on 3D-printed carriages running on the aluminium profile.

22.8 cm

X holder travel · 4,550 steps

38.0 cm

Y holder travel · 7,600 steps

200

steps per cm (X 199.6, Y 200.0)

4

limit switches, all NC

Homing drives each axis into its own switch; the corner where both are pressed is machine position (0, 0).

Z AXIS AND END EFFECTOR

LIFT, GRIP, AND A QUARTER TURN

Z axis

One NEMA17 on the moving carriage, on the 15 mm linear rail.
26.5 cm of travel, 1,350 steps.

Both ends are switches

Every build re-zeroes Z on the bottom switch and carries at the top. Neither end is a remembered number.

The claw

One servo, two positions: OPEN 0°, CLOSE 52°, on the block's 2.2 cm face. No sensor at all — hence a fixed 600 ms settle.

Rotation

A 28BYJ-48 through a ULN2003 turns the claw 512 of 2,048 steps at 10 rpm — the second block orientation.

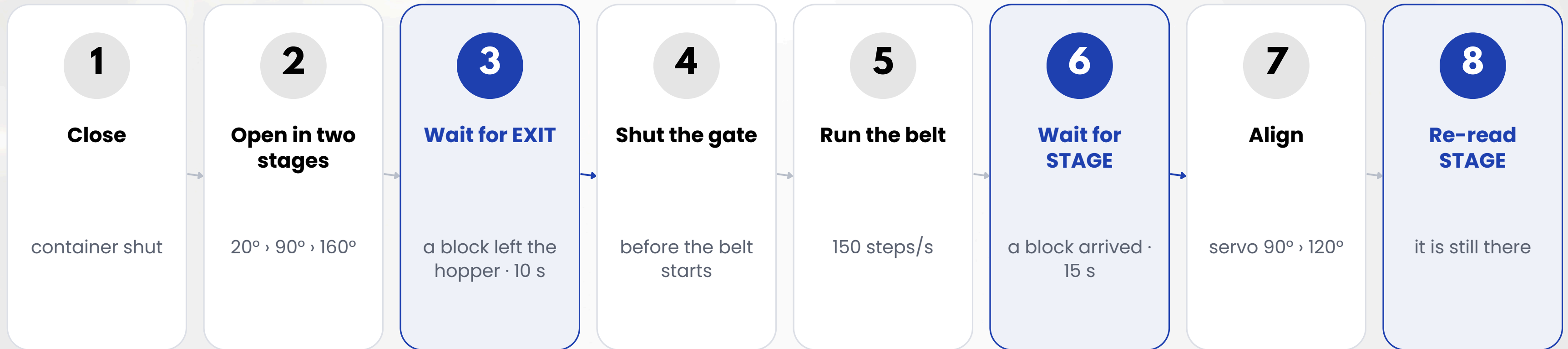


3D-printed two-jaw claw — the reference design's electromagnet is no solution for wooden blocks

The grip centre sits $(-0.3, +0.6)$ cm off the rotation axis, so a 90° turn swings the block round that axis. That swing is corrected as a per-rotation tool offset — not as a grid error.

THE FEEDER

DOSE EXACTLY ONE BLOCK — AND PROVE IT TWICE



Two ends of the feed path, three measurements — and nothing after them is assumed.

Refuses to double-load

A feed request arriving while the stage sensor already sees a block is refused with `stage_occupied` — before anything opens.

A gate becomes a doser

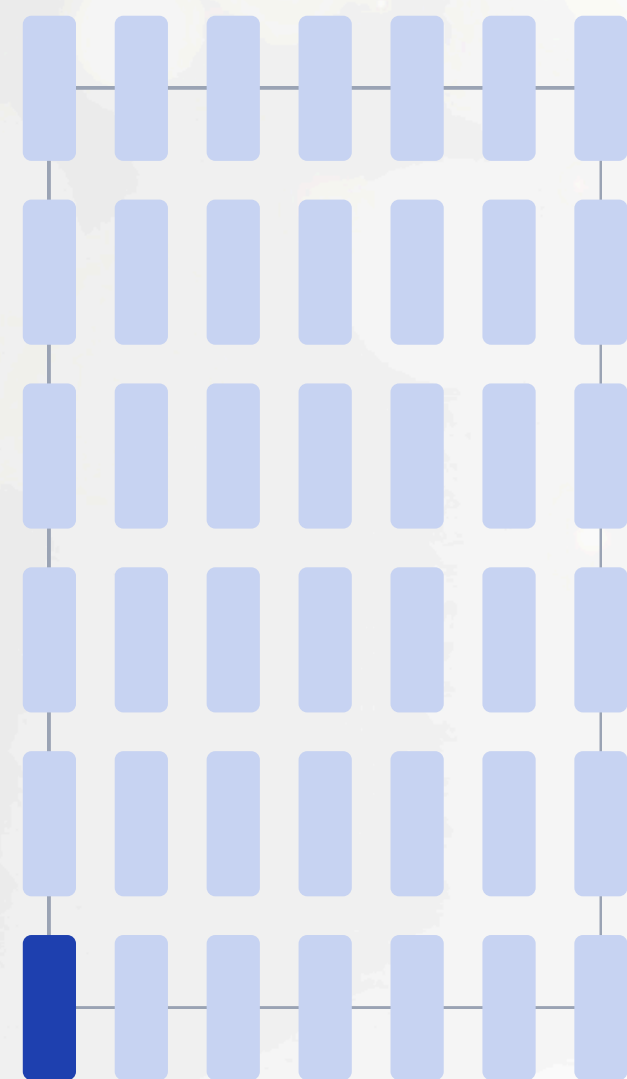
The container shuts the instant EXIT fires, before the belt starts, so a second block cannot follow the first out.

Never a success it cannot see

If the block has gone after the aligner moves, the belt resumes instead of the feeder reporting staged.

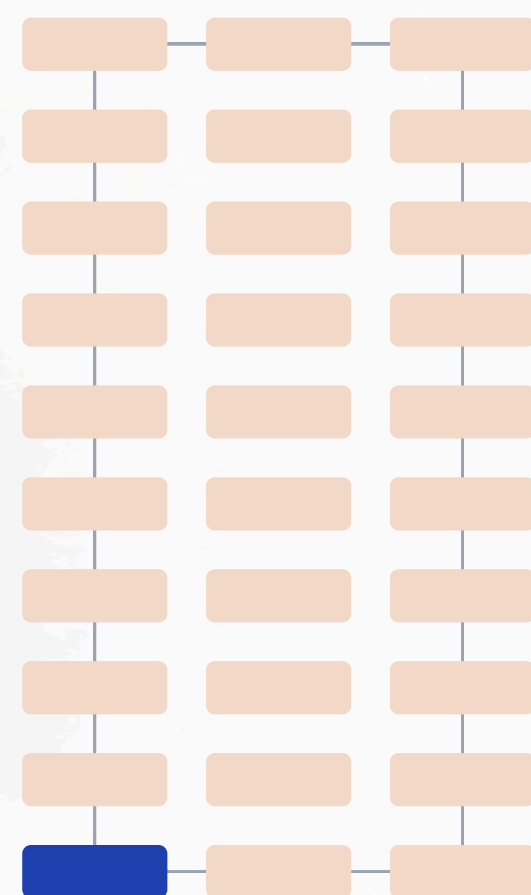
TWO GRIDS, NOT ONE GRID WITH A FLAG

ONE ENVELOPE, TWO LATTICES, ONE FEEDER CELL



VERTICAL · 2.2 × 6.0 cm

7 × 6 addressable, 41 buildable
pitch 3.8 × 7.6 cm



HORIZONTAL · 6.0 × 2.2 cm

3 × 10 addressable, 29 buildable
pitch 7.6 × 3.8 cm

One envelope

Both lattices share the same 22.8 × 38.0 cm holder travel — a block lying down does not move a limit switch.

One feeder cell

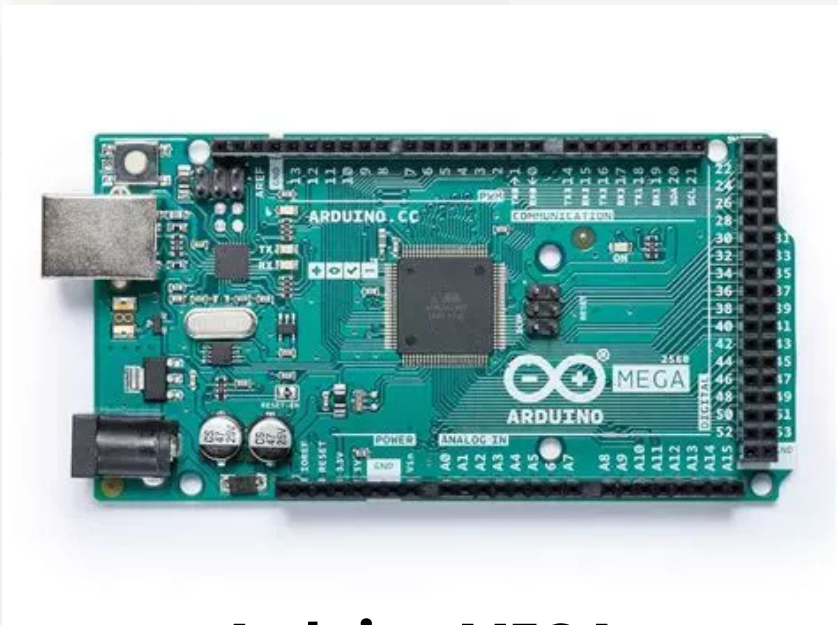
[0,0] is the feeder in both modes and is never built on: $7 \times 6 - 1 = 41$ standing, $3 \times 10 - 1 = 29$ lying.

+1.9 cm registration

$(6.0 - 2.2)/2 = 1.9$ — exactly the overhang of the rotated face, so horizontal [0,0] seats flush against vertical [0,0].

■ cell [0,0] — the feeder, never built on; its centre is the home corner

Components – Computing



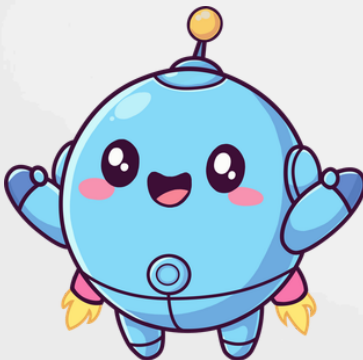
Arduino MEGA



Raspberry Pi 5



Raspberry Pi 5 Camera Module



Components – Electrical and Power



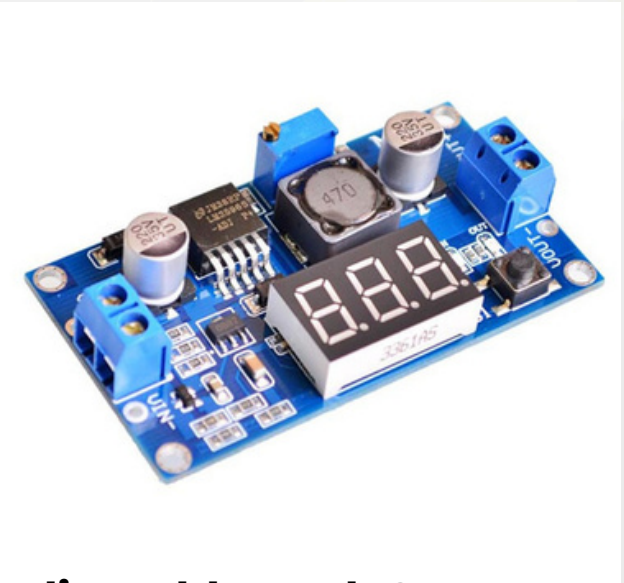
3x NEMA17 Stepper Motor



Stepper Motor 28BYJ-48



1X Servo Motor



Adjustable Buck Converter



3x TB6600 Driver



ULN2003 Driver



4x Limit Switch



Power Supply, 12V 15A

Components – Mechanical



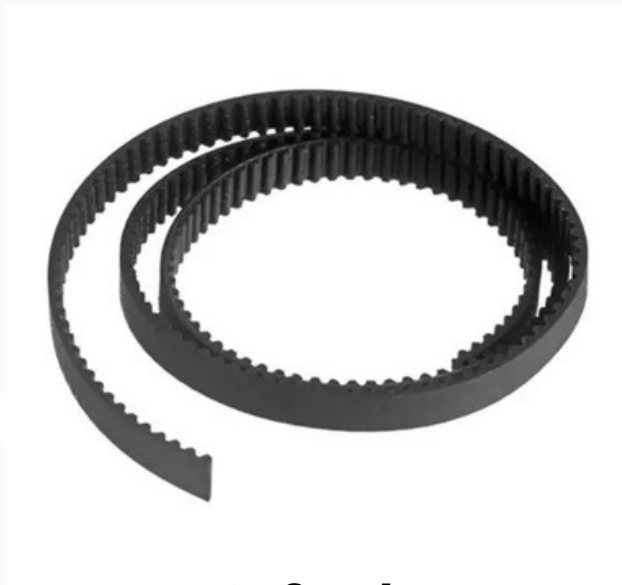
Linear Rail + Linear Motion Bearing



Timing Pulley



Aluminum Profile



GT2 Belt

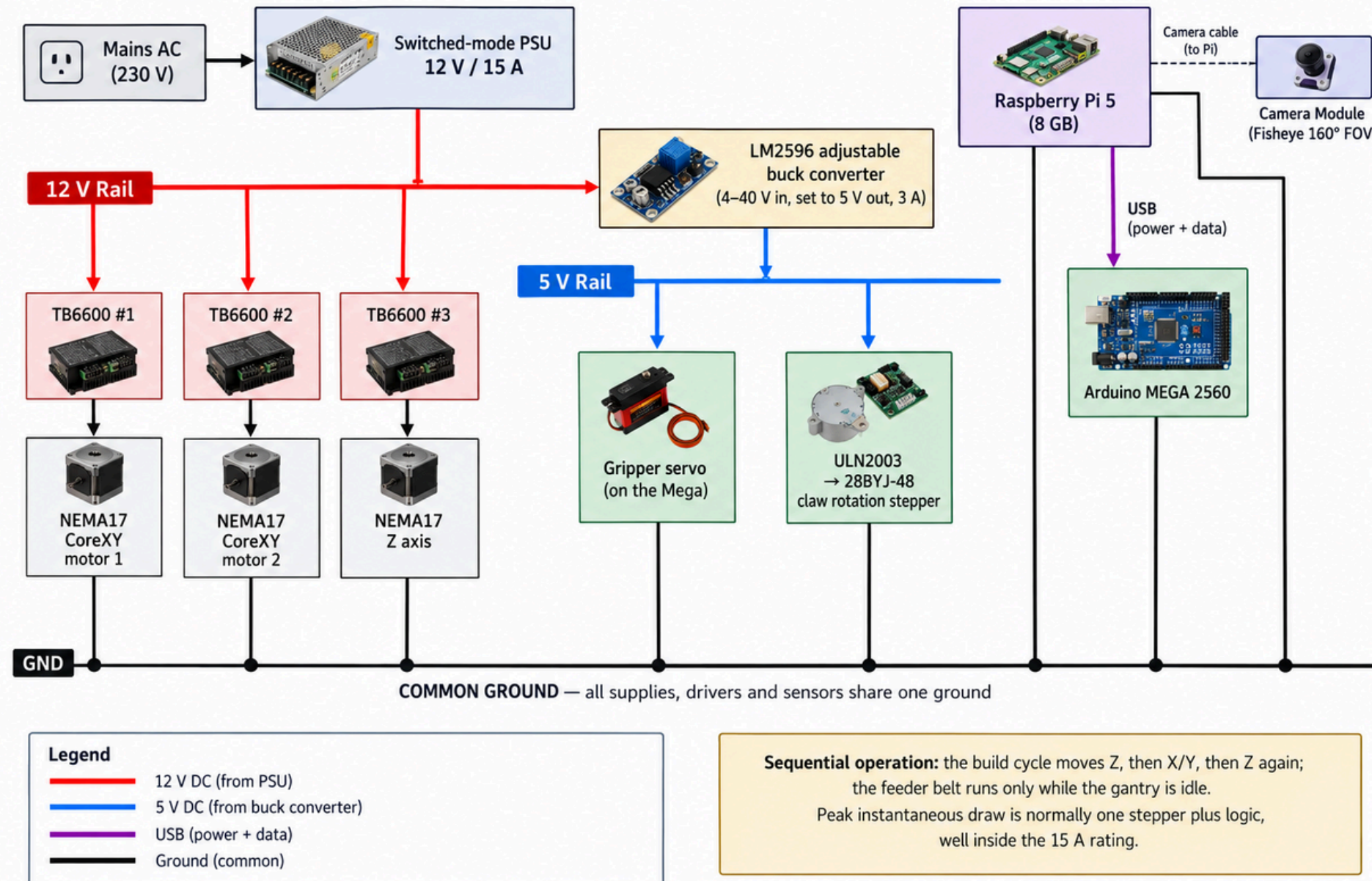


Bearing Wheels



POWER ARCHITECTURE

ONE 12 V RAIL, ONE 5 V RAIL, ONE COMMON GROUND



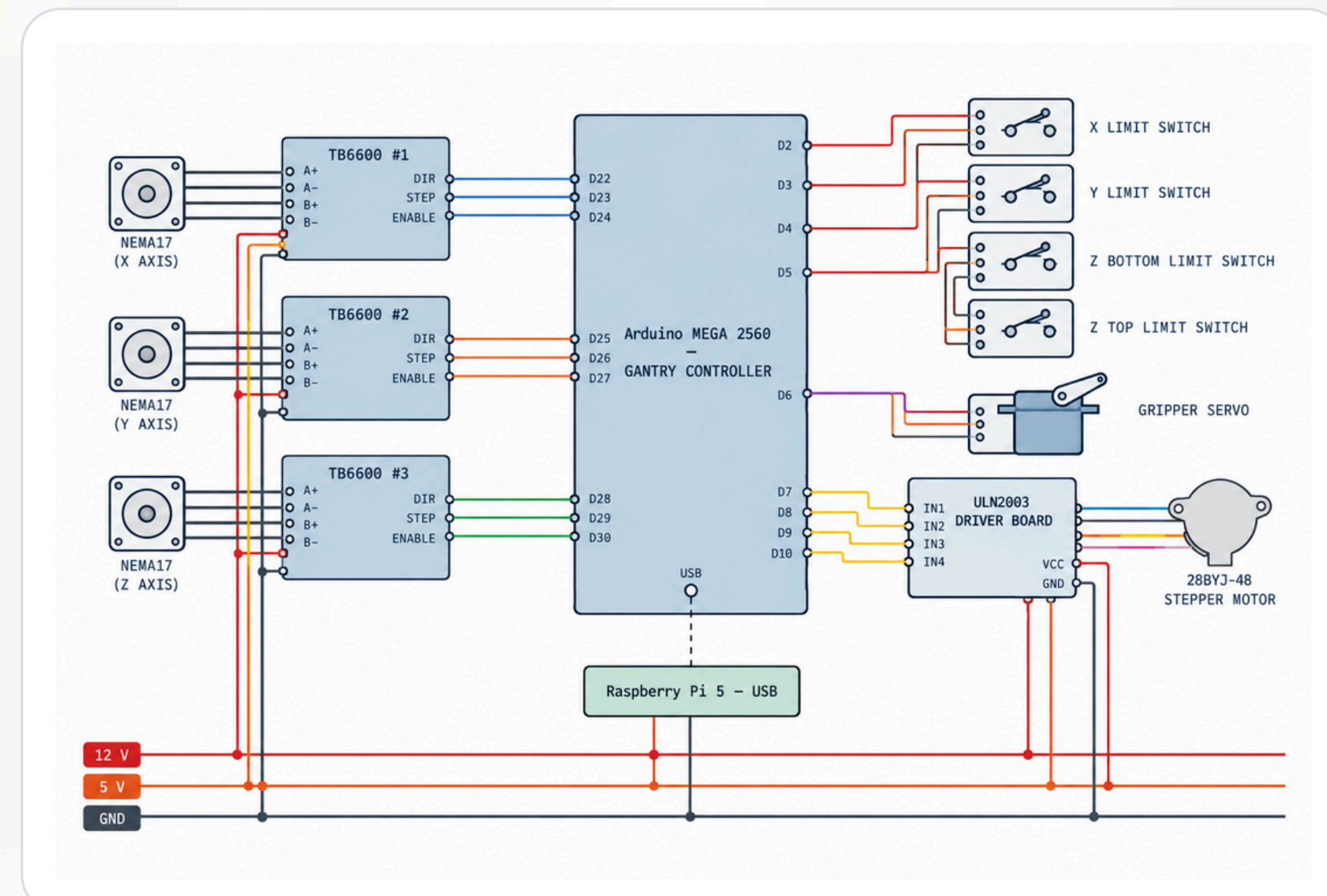
- **12 V rail** — the three TB6600 drivers and the A4988 on the feeder belt.
- **5 V rail** — an LM2596 buck converter feeds three servos, the ULN2003, both HC-SR04s and the A4988 logic.
- **Both Arduinos** — powered over USB from the Pi, which is also how they communicate.

Why 15 A is not overspecified

The build cycle moves one thing at a time, so the instantaneous draw is usually a single stepper plus logic.

WIRING

TAKEN DIRECTLY FROM THE TWO FIRMWARE SOURCES



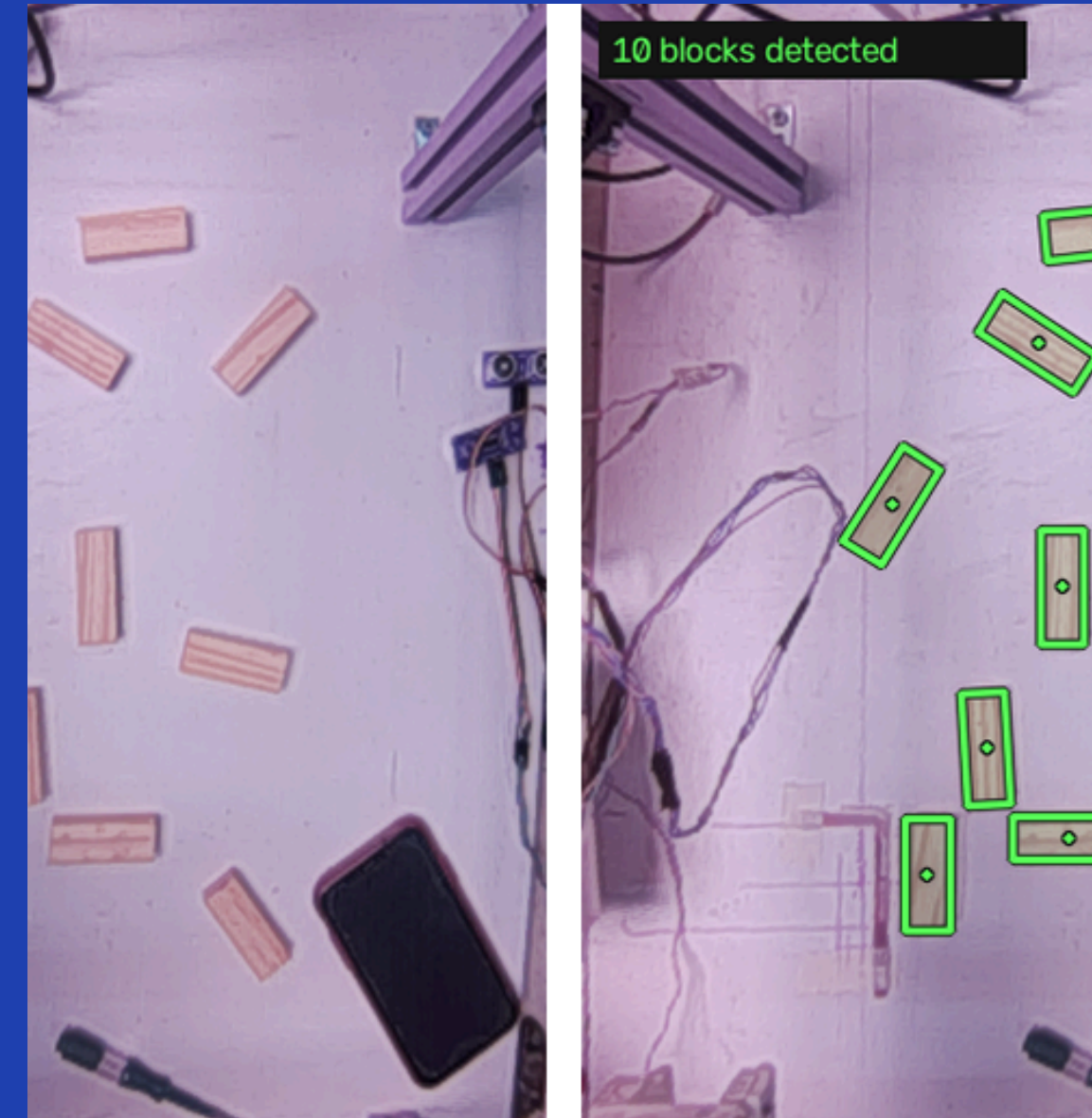
Arduino MEGA 2560 — three TB6600 drivers, four limit switches, gripper servo, ULN2003 rotation stepper

Motor and sensor runs use connectors rather than direct solder joints, so a motor can be swapped and the gantry dismantled for transport. There is no enclosure and no cable chain.

02

VISION & CONTROL SYSTEM

What the camera sees, how a pixel becomes a cell,
and the protocol that decides when the machine is
allowed to move.



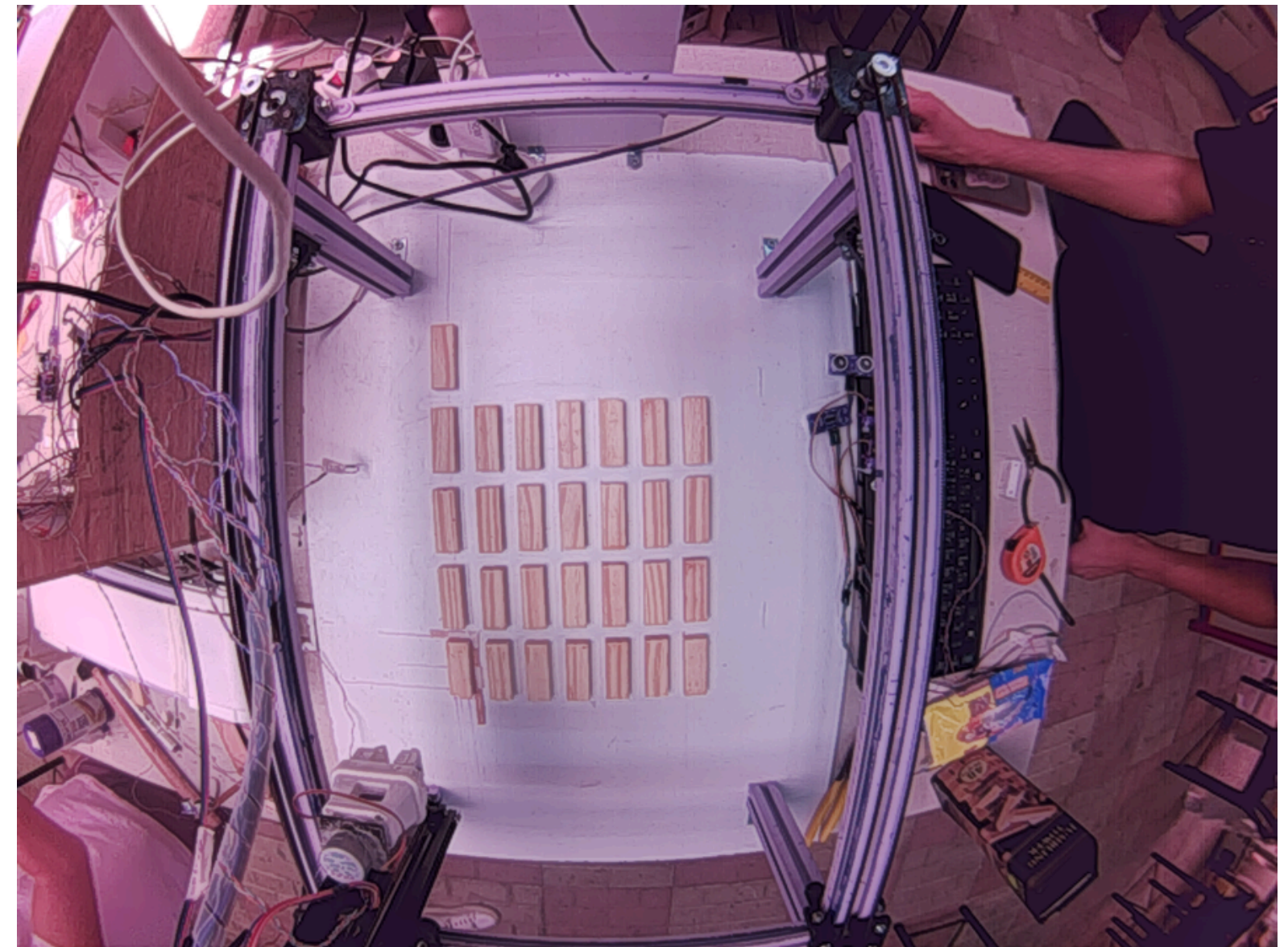
ONE LENS, TWO COSTS

WHAT A 160° FISHEYE AT 50 CM BUYS, AND COSTS

Sensor	OV5647, 5 MP, DORHEA module on the Pi 5 CSI bus
Lens	160° fisheye, mounted rigidly ≈ 50 cm above the surface
Capture	1296 $\times$ 972 through Picamera2 — the binned full-field mode
Why binned	It keeps the whole 23 $\times$ 38 cm workspace in one frame, and the per-frame cost inside budget
Lens model	Equidistant, 120° output — tuned by eye, not checkerboard-calibrated

THE TWO COSTS

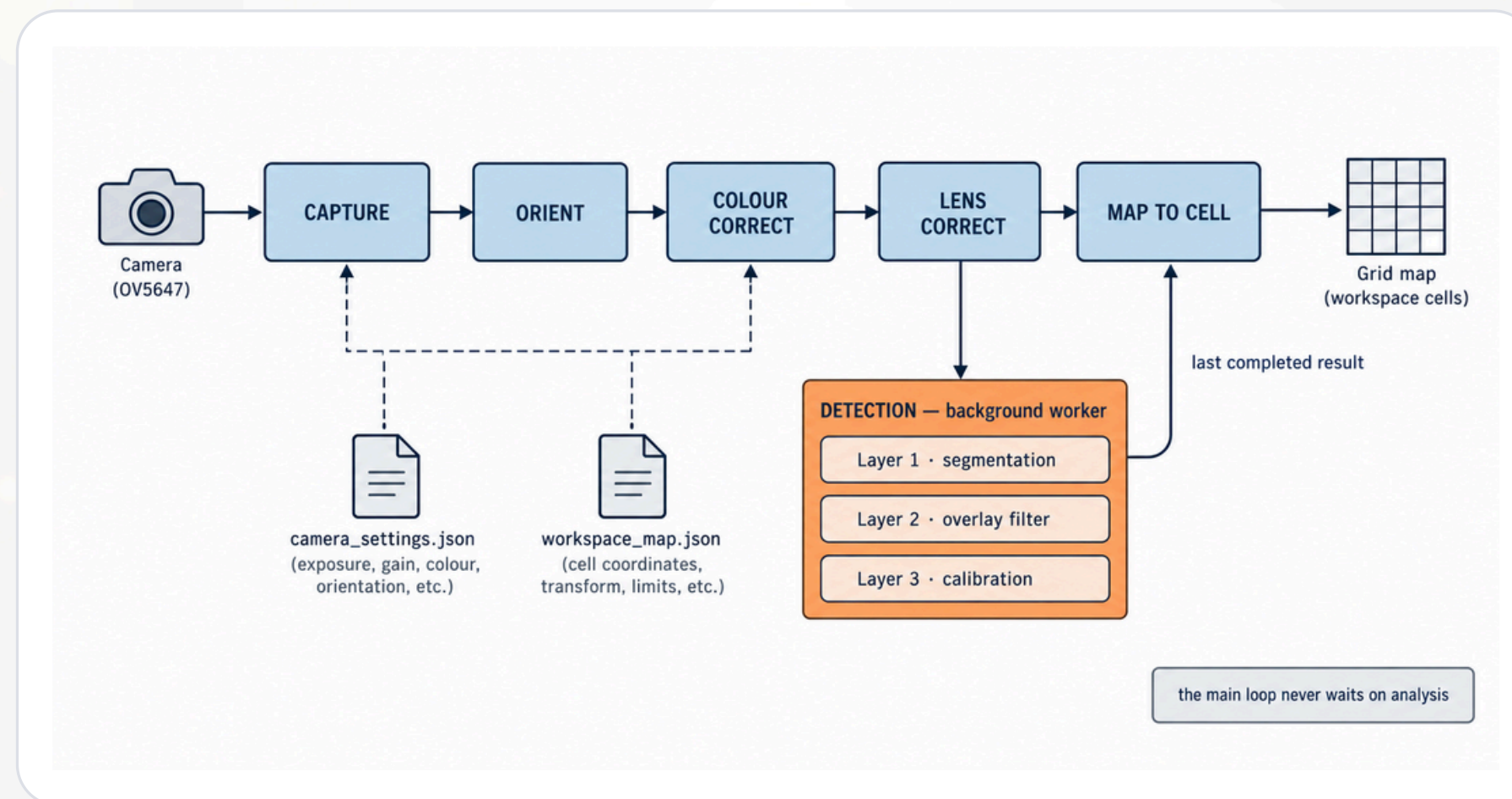
- Severe barrel distortion — the aluminium rails are straight on the machine and curved in the image
- A colour cast strong enough to be a functional fault, not a cosmetic one



A raw frame straight off the camera, before any correction

THE VISION PIPELINE

ONE PROCESS OWNS THE CAMERA; ONE FIXED ROUTE



Colour correction runs once, at the captured frame

Applied immediately after orientation and before every detector, so every tool in the system sees identical pixels. Colour is deliberately not part of the saved projection identity.

Detection runs on a background worker

At a lower rate than the video, with the main loop drawing the last completed result. A slow analysis can never stall the stream.

The Pi picks the cell; the firmware turns it into steps

The saved homography converts a pixel to physical centimetres and then to a $[col, row]$. The Pi never needs a motor step to draw or select one.

BLOCK DETECTION

A HYPOTHESIS, THEN AN ANSWER — THREE LAYERS



Ten blocks deliberately off the lattice — all ten found, everything else rejected

1

Segmentation

Warm material by red-minus-blue and red-minus-green, never by brightness — the work surface is overexposed, so a brightness cutoff selects the table and not the blocks.

2

Live overlay

Rejects, then rectifies: duplicates by IoU, boxes running off the frame (the rails), anything further than 0.34 cells from a lattice site (the holder offcuts).

3

Calibration

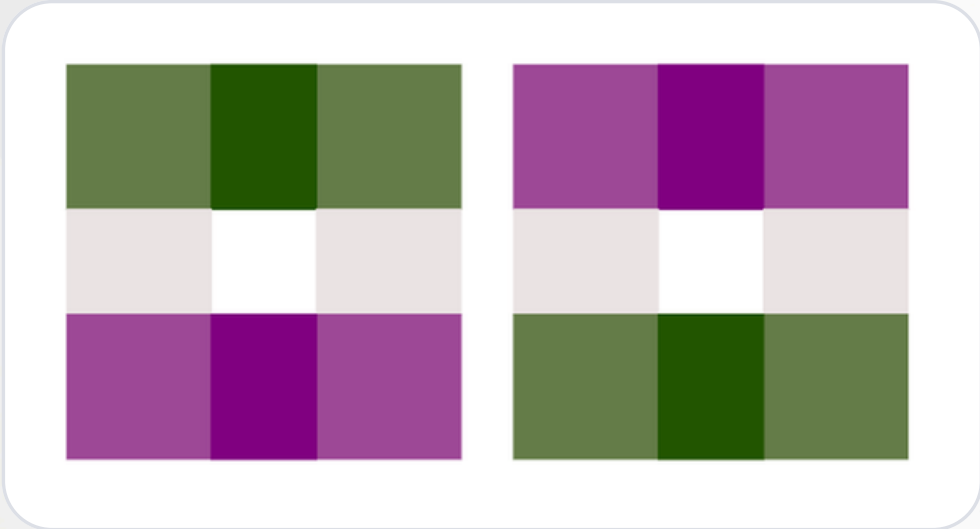
Runs once, deliberately, and is the only layer allowed to write a calibration.

TWO RULES KEEP THE OVERLAY HONEST — BOTH ASSERTED BY TESTS

- The measured centre is never snapped to the lattice — snapping would hide the misplaced block the overlay exists to show.
- The lattice filter never fires on a partial view: under six detections, or if it would reject over 30 % of what it saw, everything is kept.

GRID CALIBRATION

WHERE THE MACHINE'S GRID SITS ON THE CAMERA IMAGE



One fiducial of the printed A2 calibration target

FOUR ROUTES, ONE ARTEFACT

- **Four clicked corners** — the fast fallback.
- **Printed A2 colour target** — the primary route.
- **Evidence-assisted** — pools frames when the gantry hides part of the sheet.
- **Placed-block self-calibration** — the rig places its own blocks, so the thing measured is the thing being calibrated.

HARD ACCEPTANCE GATES, NOT STATUS READOUTS

Colour parity	every fiducial the right way round
Measured aspect ratio	against the declared geometry
Mean and maximum residual	both must pass, not just be reported
≥ 95 % physical coverage	every row and column supported
Numerical conditioning	gate at 1e-5, inside a 30-order gap

A refusal still draws what it found — green for blobs that joined a lattice, red for those that did not.

CONTROL HIERARCHY

THE PI DECIDES WHETHER; EACH ARDUINO DECIDES HOW

HIGH LEVEL • RASPBERRY PI

- Chooses the target cell, the stack level and the grid mode
- Sequences the feeder before the gantry
- Applies every safety gate and owns the session lock
- Decides whether a command may be issued at all
- Never sends a motor step, a direction bit or a delay

LOW LEVEL • MEGA AND UNO

- Step generation, direction polarity, homing, limits
- The grid-to-steps arithmetic for both modes
- Servo and stepper timing, the phase sequence
- The feed state machine, belt rate, sensor thresholds
- Everything that cannot change without reflashing

WHY THE BOUNDARY IS DRAWN THERE

The firmware owns the step caps, the Z calibration, the servo angles, the pin map and the direction polarity — physical facts nothing can push over serial, so a stale copy on the Pi would be a lie nobody notices until the rig drives into something. A handful of values genuinely exist twice, and an automated test parses the live sketch and fails if either side has moved.

THE ACKNOWLEDGEMENT PROTOCOL

ONE LINE FOR THE HUMAN, ONE LINE FOR THE PI

```
BUILD COMPLETE - block placed at [3,5] level 0  
Place time: 31.6s  
@12 OK col=3 row=5 level=0
```

Every machine line starts with @ — a character no other line in either sketch begins with, so the Pi's filter is one string test. Key=value and not JSON because on an 8 KB AVR at 9600 baud braces and quotes cost SRAM and airtime for nothing.

SAFE

Refused on validation. Nothing moved, and a retry is fine.

EIGHT KINDS • FOUR OF THEM TERMINAL

BOOT

READY

RECV

STEP

ERR

SAFE

HELD

OK

Filled = terminal. Exactly one terminal acknowledgement per command.

HELD

Failed part way through. The claw may still be gripping a block and the position is unknown. It needs a person.

They are separate kinds rather than a flag, so that Pi-side code cannot collapse them into a generic “if not ok: retry”. The retry is the thing that breaks the machine.

THE FOURTEEN-PHASE BUILD CYCLE

EVERY PHASE ANNOUNCED BEFORE IT RUNS

1	<code>raise_clear</code>	Raise Z into the top switch	8	<code>move_to_target</code>	Home, then Y to the row, X to the column
2	<code>home_feeder</code>	Home X/Y to the feeder cell [0,0]	9	<code>rotate_to_grid</code>	Apply the active grid's rotation
3	<code>neutralise_claw</code>	Return the claw to neutral	10	<code>lower_to_level</code>	Lower Z to the target level
4	<code>open_claw</code>	Open the jaws, still clear above	11	<code>release</code>	Open the claw. The block is placed
5	<code>lower_to_ground</code>	Lower Z into the bottom switch — re-zeroes Z	12	<code>park_clear</code>	Raise Z clear of the block just placed
6	<code>grip</code>	Close the claw. The block is held	13	<code>park_home</code>	Return X and Y to the origin
7	<code>lift_block</code>	Raise Z to carry height	14	<code>park_rotation</code>	Return the claw to neutral

Phases 1–11 place the block; 12–14 park the rig. Parking is not decoration — placed but not parked is not a success, and the terminal result is downgraded from OK to HELD.

ONE MESSAGE AUTHORISES MOTION

THE TWO-BOARD HANDOFF, AND THE FIVE LAYERS ABOVE IT

THE HANDOFF

Pi › Uno FEED <id>

Uno › Pi @id RECV / ACK / STATE / SENSOR / EVENT

Uno › Pi @id OK state=block_ready result=staged

Pi › MEGA B <col> <row> <level>

MEGA › Pi @seq STEP step=n/14 phase=... status=begin

MEGA › Pi @seq OK col=... row=... level=...

Only that one terminal message, with a matching id, authorises the B.

- ACK, STATE, SENSOR and EVENT are progress, never success. A terminal with the wrong id is ignored.
- After the B is sent, even a SAFE rejection locks the session — a block is already staged.

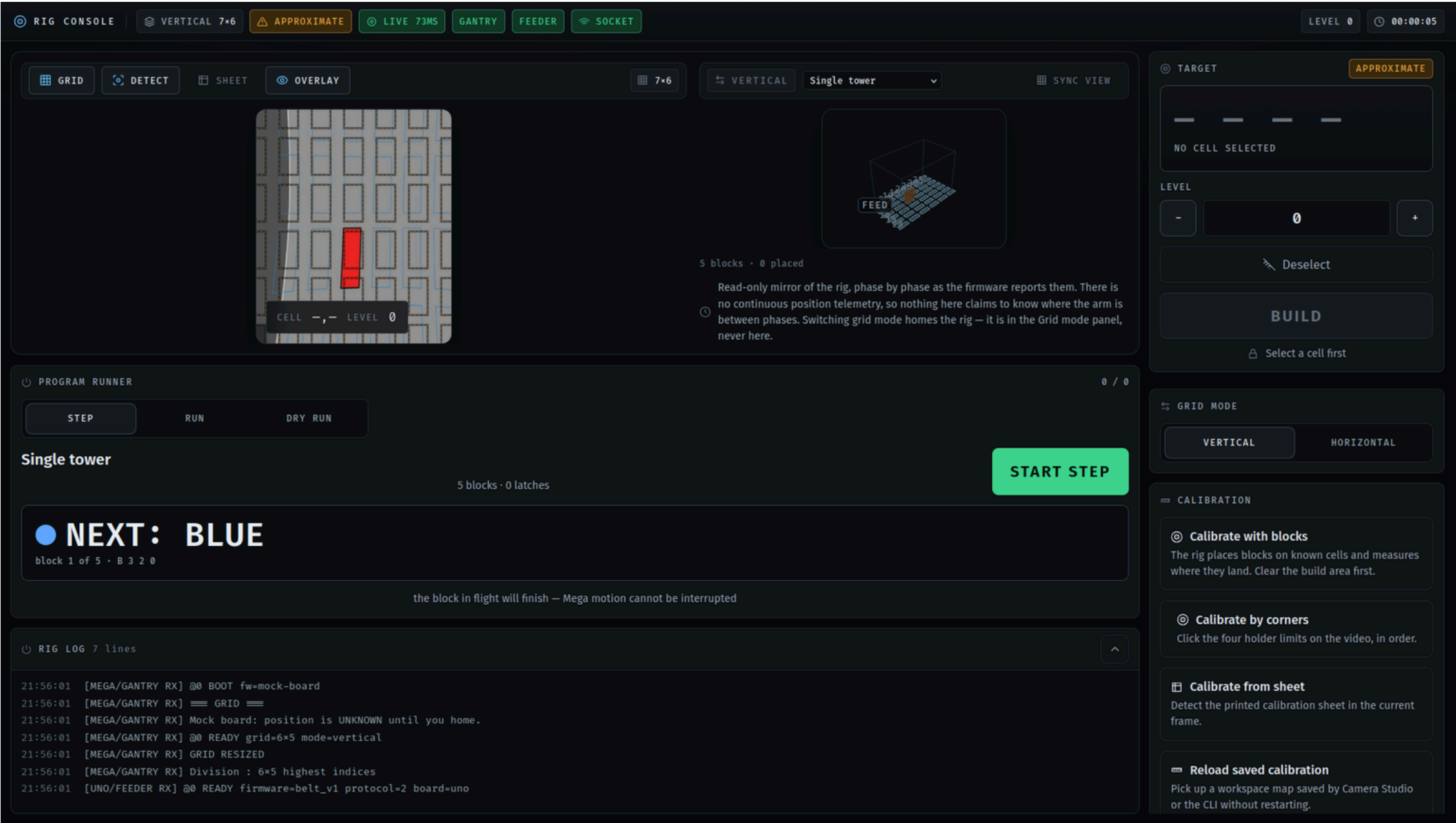
THE GUARD STACK

- 1 HTTP route**
unconfirmed request · mismatched command · stale frame · locked session
- 2 Build worker**
a second build already running
- 3 Safety state**
a locked session, or no cell selected
- 4 Two-board handoff**
another placement already owns the pickup point
- 5 Serial clients**
an overlapping transaction, or the wrong board

The browser is a mirror — a greyed-out button is never the safety mechanism.

THE OPERATOR CONSOLE

LIVE CAMERA, TAP A CELL, TWO TAPS TO MOVE



Tap a cell.

Selection moves nothing and shows the exact command, B 3 2 1.

Tap BUILD, then CONFIRM.

One correlated FEED, then the B. Two deliberate taps.

Three outcomes.

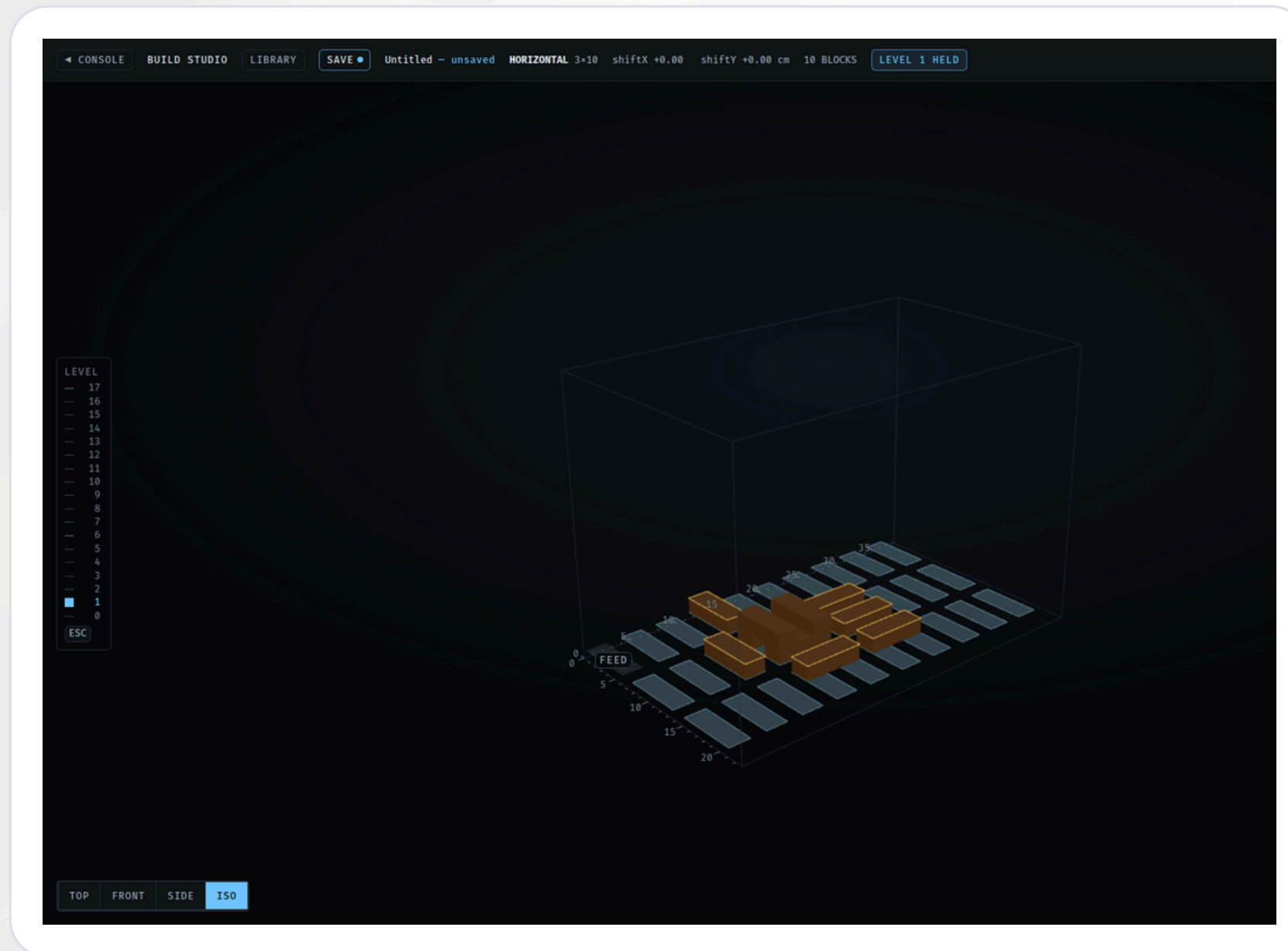
PLACED (green, selection cleared), REJECTED (amber, nothing moved), ABORTED (red, session locked).

No cancel, no retry.

There is no such control, because the machine cannot honour either.

THE 3D BUILD STUDIO

DESIGN, VALIDATE, COMPILE — THEN RUN IT



THE VALIDATOR ASKS THREE QUESTIONS

- **Support** — a centre-of-mass toppling test, not a contact-ratio shortcut.
- **Collision** — in machine space, not grid indices, so a horizontal block spanning two columns is checked correctly.
- **Reachability** — inside the active grid, under the firmware's build ceiling.

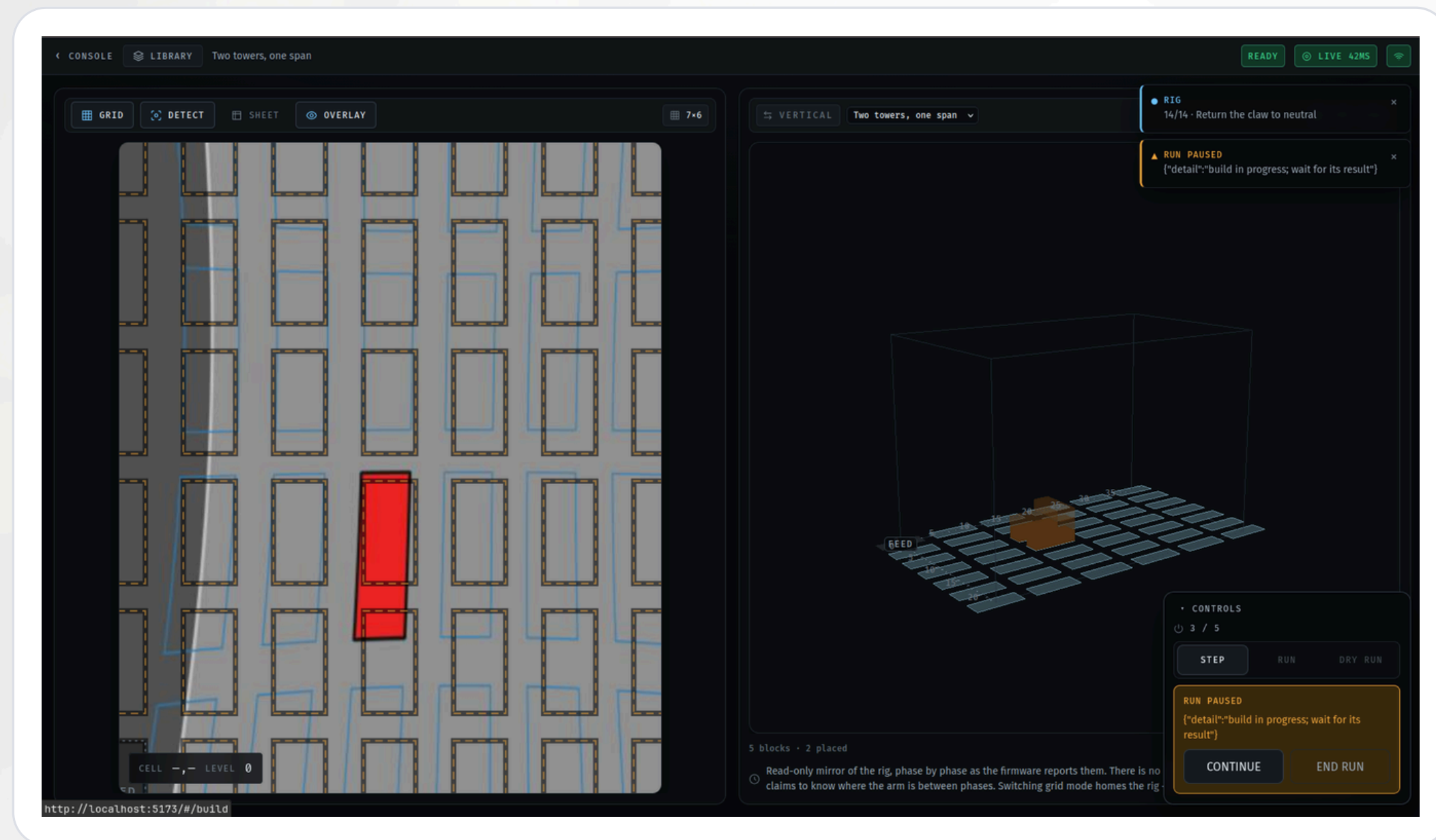
THE COMPILER RUNS IN FOUR STEPS

- **Support graph** — which blocks rest on which, from footprint overlap.
- **Ordering** — Kahn's algorithm, bottom-up, grouping same-orientation runs to minimise mode latches.
- **Emission** — a mode command only on an actual change.
- **Summary** — blocks, latches, levels and a time estimate.

Deterministic — twenty compiles, byte-identical output. An invalid model compiles to nothing at all, never to a half-program.

THE LIVE DIGITAL TWIN

IT DRAWS WHAT THE FIRMWARE SAID — AND NOTHING ELSE



It never invents state.

A block is drawn as placed because the server reported a terminal placed result for it, and for no other reason.

After an abort it stops.

A locked session freezes the animation, desaturates every block and demotes everything unconfirmed to a ghost.

Phase-driven, not clock-driven.

An earlier version interpolated a 1.6 s descent from the browser's clock and looped it — a 40 s build showed twenty-five descents that never happened.

If Z jams,

the block glides down, stops just short of the cell and sits there visibly not landing — which is the truth.

03

TESTING & RESULTS

Sixteen complete builds on hardware, two reference boards, and 560 automated tests.



TESTING METHODOLOGY

EVERY QUESTION ANSWERED AT THE CHEAPEST HONEST LEVEL

1

Automated software tests

no hardware

Grid arithmetic, protocol parsers, the compiler, the twin, the homography. The Python and TypeScript grids are held to each other by shared fixtures.

2

Protocol-level simulation

no hardware

A fake MEGA and a fake Uno speak the same acknowledgement grammar, including failure, reset and cancellation. This is how the two-board handoff was tested exhaustively.

3

Firmware host builds

no hardware

A stub-Arduino harness proves the sketch parses and that its output is byte-for-byte the protocol. A clean compile is not treated as a test of behaviour.

4

Hardware runs on the rig

the real machine

Anything touching motion, limits or Z. Every run appends to two logs — a stopwatch per build, and every serial line with the gap since the last one.

The rig takes half a minute to answer one question, and a wrong answer can drive a claw into a stack — so the logs from level 4 are the primary evidence, and everything else is rehearsal.

CYCLE TIME AND REPEATABILITY

SIXTEEN BUILDS ON HARDWARE, ALL SIXTEEN PLACED

26.1 s

mean cycle, per block

19.8 – 32.2 s

fastest and worst case

< 45 s

the objective

12.57 s

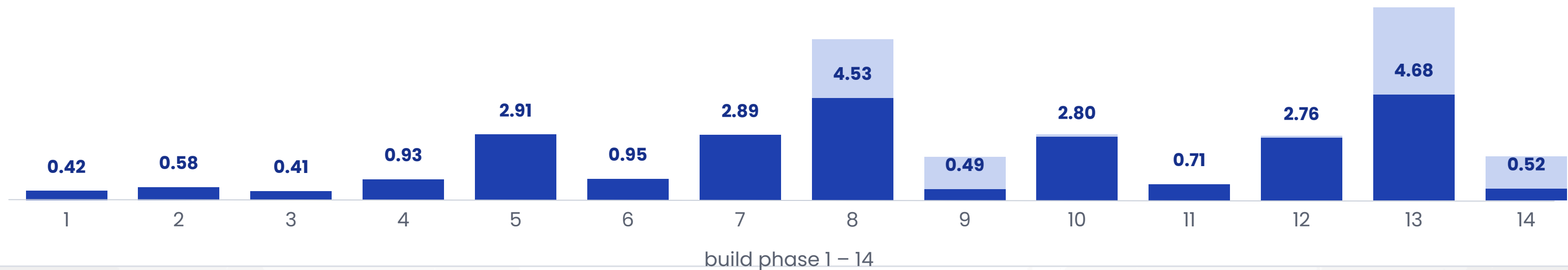
the nine fixed phases

0.18 s

their standard deviation

Mean duration of each build phase, over sixteen builds (seconds)

Light bar = minimum to maximum across all sixteen · solid bar = mean



Nine phases never move

Nine of the fourteen have no distance or level dependence at all. Across all sixteen builds they summed to 12.57 s ± 0.18 s — six of them identical to the 0.01 s logging resolution.

The variation is travel, and nothing else

Only phases 8 and 13 range widely, 1.48 s to 8.53 s. With no acceleration ramp, a move of N steps takes N times a fixed period.

VISION RESULTS

TWO REFERENCE BOARDS, TWO NEGATIVE RESULTS

29 / 29

blocks found, both boards

33 › 29

hypotheses after rejection

37 – 103 ms

per analysed frame

0.85 px

mean fit residual

0.27 cm

mapping error on a 2.2 cm
block



29 blocks, the rails, and two wooden
offcuts that are not blocks

WHAT THE FIT CHOSE, AND WHY

- **Chosen on held-out prediction, not training error.** Similarity 101.4 px, affine 1.38, homography 1.22, homography + curvature 1.09.
- **Pitch is static to within 2 %.** X 29.38 px (sd 1.24), Y 69.14 px (sd 1.08).
- **An independent geometry check.** The view is 17.7 % anisotropic; two independent estimates agree to 4 %.

The 0.27 cm is the saved four-corner format flattening the fit — not the fit itself.

TWO NEGATIVE RESULTS

Higher resolution finds nothing extra

29/29 at every width from 384 to 1024 px.
Illumination flattening costs up to 3,894 ms a frame and finds exactly the same 29.

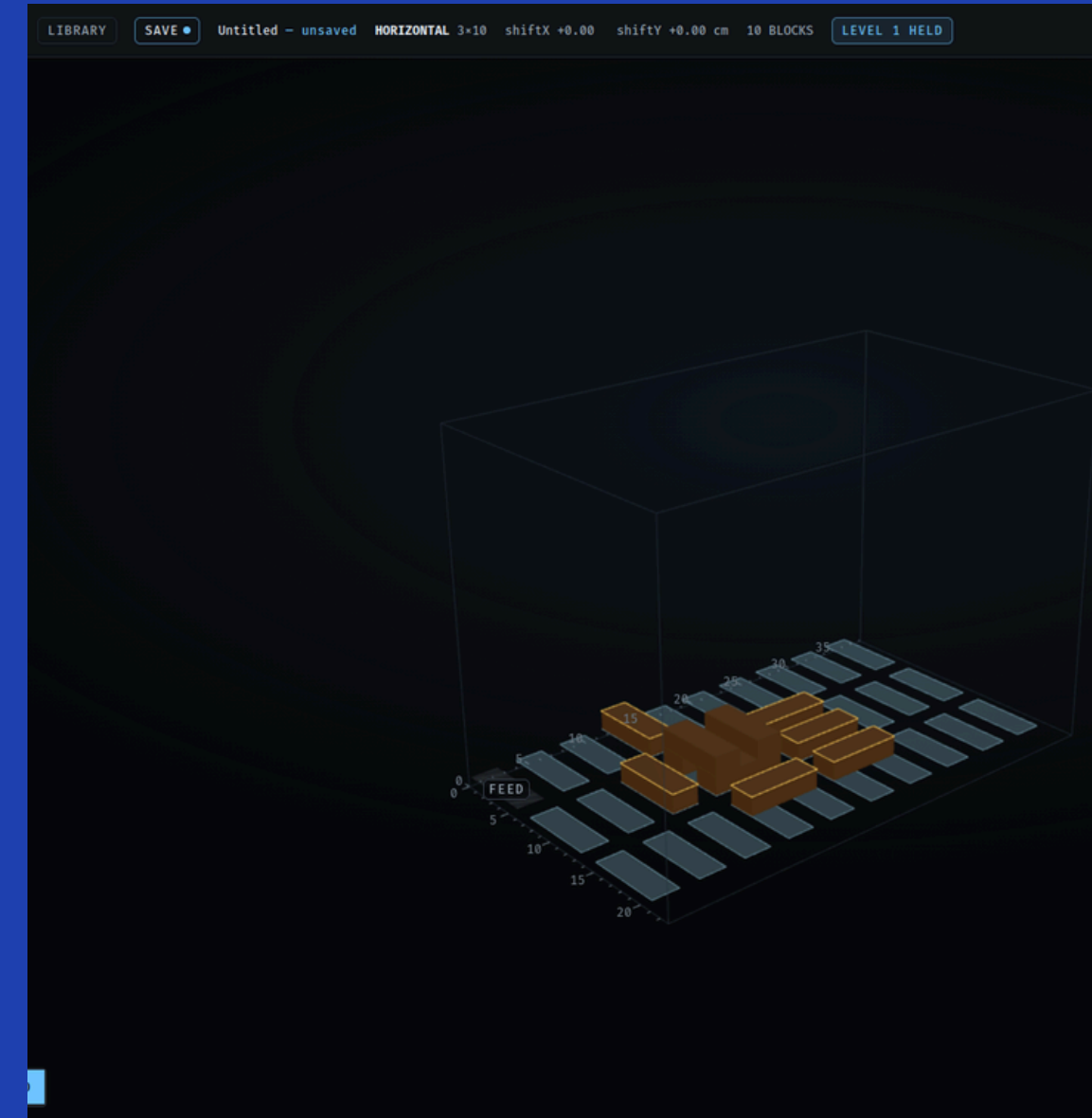
White balance is right for the sheet, wrong for the board

On the printed sheet it takes the green mask from 0 pixels to 87,836. On a board of blocks it pulls the wood toward the surface: 28 of 29 with it on, 29 with it off.

04

CONCLUSION & FUTURE WORK

What the machine proves, the limits that are real, and the one item on the list that is not optional.



LIMITATIONS, STATED PLAINLY

LIMITS OF THE HARDWARE, NOT OF THE APPROACH

No hardwired emergency stop

Nothing removes motion energy independently of the software. The stop of last resort is the supply. The machine is run attended.

Motion cannot be interrupted

The gantry does not read serial while it moves, so software stop is stop-after-current-block by construction.

No watchdog

If the Pi crashes or the cable is pulled, the MEGA finishes the command it already holds.

Open-loop steppers, no stall detection

No encoders, no current sensing. A stall produces step pulses that go nowhere.

The lens is tuned, not calibrated

Adjusted by eye against straight edges — visually straight, not measurement-grade. The file records source: estimated.

The claw angle is not sensed

The operator is trusted to start neutral. If not, every placement that session is turned 90°.

And the layer that would let the camera notice a block that never arrived — placement supervision — is fully designed and not built.

FUTURE WORK

ONE ITEM IS NOT OPTIONAL; THE REST ARE

SAFETY, FIRST

- A hardwired emergency stop — a latching mushroom button in series with a contactor on the 12 V motor rail
- A stop that is observable during motion, and a communications watchdog that brings the rig to a defined state

Close the loop on placement

- A server-side ledger of what went where
- Compare at the park state — a known, repeatable position
- A pick-from-cell verb, so the machine can act on a wrong placement

Measure properly

- A checkerboard or ChArUco lens calibration
- A per-cell table in the saved map, removing the 0.27 cm flattening error
- Fix the camera, not the matrix — the colour correction is at its limit

Protocol and capability

- Controller-supplied command ids
- Encoders or stall detection — also worth about a third of the cycle time
- A faster serial link, and a larger workspace

CONCLUSION

THE CONTRIBUTION IS NOT THE GANTRY

Cartesian gantries are a solved problem. What this project demonstrates is the layer student-scale automation usually leaves out: every physical transition confirmed by something other than a clock.

The feeder

reports success because a sensor saw a block — and saw it again after the aligner moved.

The gantry

narrates each of its fourteen phases before running it.

The twin

draws what the firmware said, and stops short of the cell until the release event arrives.

And when it cannot prove

what happened, it locks and asks for a person rather than retrying.

16 / 16

hardware builds placed

26.1 s

mean cycle

5

levels on one cell

29 / 29

blocks detected

560

automated tests

THANK YOU

QUESTIONS

Ahmed Gharib · Mohie Halawa · Khalil Qanabita

Supervised by Dr. Samer Arandi

Department of Computer Engineering · An-Najah National University

